



6.1 Student Guide: Advanced Bash

Lesson Overview

Today's class will introduce you to combining commands, making custom commands, and Bash scripting.

For the next two classes, you will work through a series of exercises in which you will create custom commands and Bash scripts to collect evidence, audit, and reconfigure a Linux machine and take steps to harden the system.

Lesson Objectives

By the end of this lesson, you will be able to:

- Construct compound commands using `&&`, `|`, and file redirects.
- Create and save alias commands to their `~/.bashrc` file.
- Edit `$PATH` variables to include a custom `~/scripts` directory.
- Create simple Bash scripts that consist of a list of commands.

Module 6 Day 1 Contents

Slideshow

The slides for today can be viewed on Google Drive here: [6.1 Slides](#).

- **Note:** Editing access is not available for this document. If you wish to modify the slides, please create a copy by navigating to File > "Make a copy..."

01. Compound Commands

Today we will cover:

- Creating compound commands by chaining several commands together;
- Creating custom commands using aliases;
- Creating short Bash scripts; and
- Creating custom commands from Bash scripts.

We will start with creating compound commands.

Creating Compound Commands

Using compound commands at the command line is a fundamental IT skill and essential to the role of a sysadmin.

- Navigating Linux directories, quickly searching large log files, and writing small scripts to automate tasks will save you time and energy.
- Because Linux is used widely throughout the security field, these same skills will prove invaluable in many security roles.

A **compound command** is several commands that are normally written out one-by-one, chained together in order to create a new command.

- We can also think of it as chaining together different programs to accomplish a given task.

For example: `file $(find / -iname '*.txt' 2>/dev/null) > ~/Desktop/text_files`
`; tail ~/Desktop/text_files`

- This command does the following:
 - Searches the entire computer for files ending in `.txt`.
 - Verifies that the files found are text files.
 - Ignores any errors it comes across.
 - Creates a list of all found files before saving that list to the desktop.
 - Finally, it will open the file and print the last 10 lines that were added.

By the end of this lesson, you should be able to describe the function of each character in these types of commands, improve the syntax of the command, and create new commands to accomplish new tasks.

Breaking Down the Command

You should be familiar with the three components of a command:

1. The **program**, which is a binary program that you are running.
2. The program's **options**, which change the behavior of the program being run in the first part of the command.
3. The **arguments** provided to the program, usually a reference to a directory or file that you want the program to act on.

Commands typically follow this format:

```
program -options arguments
```

In this case, the `arguments` part of the command is acting as input for the command.

- However, when you start chaining commands together, the output of one command becomes the input of the next, and so the `argument` is only needed for the first command in the chain.

Compound commands typically follow this format:

```
program -options arguments | program -options | program -options | program  
-options
```

- You've already chained commands together using various techniques like `>`, `>>`, and `|`.

Chaining with `>` and `>>`

```
ls > list.txt
```

- This command takes the output of the `ls` command and sends it into a new file named `list.txt`. If the file `list.txt` already exists, it is overwritten with the output of the `ls` command.

```
> list.txt
```

- In this case, we didn't put a command in front of `>` so that there is no output to send to the `list.txt` file.
- However, the file is still written just with no output, so a blank file is written. If the file `list.txt` exists, it is overwritten with nothing.

```
ls >> list.txt
```

- `>>` will append the output of the `ls` command to the `list.txt` file.
- If the `list.txt` file does not exist, it is created.
- Therefore, using `>>` instead of `>` is always safer, unless you want the file to be overwritten.

Piping with `|`

```
ls -l | grep '.txt'
```

- The pipe (`|`) takes the output of one command and sends it to the input of another command.
- Compound commands with pipes typically follow this format:

```
program -options arguments | program -options | program -options |  
program -options
```

 - `ls -l` creates a list of files.
 - `|` pipes the list from `ls` into the command that follows.

- `grep` searches the files from `ls` for the string that follows.
- `.txt` matches any file that contains `.txt` in the filename.

Some other common programs users pipe to:

- `| head` prints only the first 10 lines of output.
- `| tail` prints only the last 10 lines of output.
- `| sort` sorts the output alphabetically.
- `| sed` searches and replaces parts of the output.
- `| awk` displays only specified parts of the output.

Note this advanced example of chaining commands together using `|`:

```
cat /etc/passwd | grep sysadmin | awk -F ':' '{print $6}'.
```

- `cat /etc/passwd` dumps the contents of `/etc/passwd` to output.
- `|` pipes that output into the command that follows.
- `grep sysadmin` displays lines that contain `sysadmin`.
- `|` pipes that output into the command that follows.
- `awk -F ':' '{print $6}'` prints only the sixth field of the line.
- `awk` usually looks for a space to use as a field separator, but in this case, we want it to separate the line by a colon, because `/etc/passwd` uses colons to separate its fields.

Combining with ;

We can also use a `;` to run a series of commands back to back.

When using `;`, each command is running on its own. It is not sending its output to the next command. Therefore, each command can have its own arguments.

- For example, rather than typing this:

Unset

```
$ mkdir dir
$ cd dir
$ touch file
$ ls -l
-rw-r--r-- 1 user user 0 Sep  4 15:33 file
```

- We can use one command:
 - Run `cd ...`

- Run `rm -r dir`.
Remember from earlier modules that `rm -r` will remove a directory and the files contained in it.
 - Type `mkdir dir; cd dir; touch file; ls -l`
- Each command will happen in succession.
 - First, the `mkdir` command, then `cd`, `touch`, and finally `ls`.

Compound commands using `;` typically follow this format:

```
program -options arguments ; program -options arguments ; program -options arguments ; program -options arguments
```

- For example: `mkdir dir; cd dir; touch file; ls -l`
- The output would be:

Unset

```
$ mkdir dir; cd dir; touch file; ls -l
-rw-r--r-- 1 user user 0 Sep  4 15:33 file
```

`;` will run each command back to back, no matter the outcome of the commands. Therefore, using a `;` to chain commands together may not always give you the correct outcome.

If we removed the files you just created:

- Run `cd ...`
- Run `rm -r dir`.
- Type `mkdir dir; cd dor; touch file; ls -l`.
 - This command will fail because you are trying to move into the directory `dor`, which has not been created. However, the commands `touch` and `ls` will still run.
- Run `mkdir dir; cd dor; touch file; ls -l`.
Your output should be similar to:

Unset

```
-Bash: cd: dor: No such file or directory
drwxr-xr-x 2 user user 4096 Sep  4 15:52 dir
-rw-r--r-- 1 user user 0 Sep  4 15:52 file
```

Notice the error reported for `cd`.

- We still have a `file` and the `ls` command to run, but we did not get our desired output because we were not in directory `dor`.

Combining with &&

A better operator to use in the previous case is the `&&`. The `&&` will run the next command *only* if the first command was successful.

```
mkdir dir && cd dir && touch file && ls -l
```

- If the command was written this way, `cd` would only run if `mkdir` was successful, `touch` would only run if `cd` was successful, and `ls` would only run if `touch` was successful.

Compound commands using `&&` typically follow this format:

```
program -options arguments && program -options arguments && program  
-options arguments && program -options arguments
```

- In this case, the only commands that run are `mkdir dir` and `cd dor`. `cd dor` fails, so `touch` and `ls` are ignored.
- `mkdir dir && cd dor && touch file && ls -l`

Your output should be similar to:

```
Unset
```

```
-Bash: cd: dor: No such file or directory
```

- Run `ls` to show that only `dir` was created.

Section Summary

- `>` to create files with the output of a command.
- `>>` to append the output of a command to a file. Creates a file if the file does not exist.
- `|` pipes the output of one command into another command.
- `;` to chain commands together in succession.
- `&&` to chain commands together. The second command runs only if the first command was successful.

02. Compound Commands Activity

- [Activity File: Compound Commands](#)
-

03. Review Compound Commands

04. Creating Aliases

Compound commands are useful but do require a lot of typing. If you use a compound command often, it might be nice to save it somewhere so that you can easily reference it.

An **alias** is a shorthand or custom command that you can define, which will launch any command or compound command, including arguments and redirection.

Next, we will create custom commands using aliases and save them into a configuration file so that they are available every time you log in.

- System administrators commonly use custom commands in everyday work to save time.
- We are going to use them to make our audit commands even easier to remember and use.

NOTE: Many Linux distributions have aliases installed by default.

- `ls` by default shows the files and directories in the current directory in white.
- In our system, there is already an alias for `ls` that outputs `ls --color=auto`, which shows files in white, directories in blue, and executables in green.

Aliases Demo

- Log into the lab environment with the username `sysadmin` and password `cybersecurity`.
- Open up the Terminal.

The syntax for creating an alias is as follows:

- Type `alias lh='ls -lah'`.
 - `alias` indicates that we are creating an alias.
 - `lh` is our custom command we will use to store the command we want to run.
 - `ls -lah` is the command that will run when we use our alias `lh`.
- Run `alias lh='ls -lah'`.
- Run `lh`.

Output should resemble:

Unset

```
$ alias lh='ls -lah'
$ lh
```

```
total 52K
drwxr-xr-x  6 user user 4.0K Sep  4 16:00 .
drwxr-xr-x 28 user user 4.0K Aug 27 14:46 ..
drwxr-xr-x  3 user user 4.0K Aug 28 12:51 1
drwxr-xr-x  3 user user 4.0K Aug 28 12:52 2
drwxr-xr-x  2 user user 4.0K Aug 27 14:46 3
drwxr-xr-x  2 user user 4.0K Sep  4 16:00 dir
```

Now we can use `lh` any time we want to run the command `ls -lah`.

Note: If we use for our alias a command that already exists, we will change the way that command behaves, or possibly stop that command from working.

- For example: If we wanted the `ls` command to *always* default to `ls -l`, we could create an alias to override the `ls` command.
- Run `alias ls='ls -l'`.
- Run `ls` to show that the behavior has changed.

We can see a list of all the aliases we currently have access to by simply typing `alias`.

- Run `alias`.
Your output should be similar to:

Unset

```
$ alias
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias ls='ls -l'
alias ll='ls -aF'
alias la='ls -A'
alias l='ls -CF'
alias lh='ls -lah'
```

All the aliases listed are configured by default. We will discuss how to change them in a moment.

If you want to remove an alias, you can use the `unalias` command.

- Run `unalias ls`.

- Run `alias`.
Your output should show that the `ls` alias has been removed.

Unset

```
$ unalias ls
$ alias
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'
```

- Run `ls` to show that the expected output has returned.

Aliases created in this way only work for the session in which you have created them. So, once the terminal is closed and re-opened, the alias will be gone.

- Close and reopen the terminal.
- Run `lh`.
You should get:

Unset

```
-Bash: lh: command not found
```

Keeping Aliases Across Sessions and Logins

If we want these commands to be available every time we login, we need to store them in a configuration file that loads every time we open a terminal.

The terminal has several configuration files, but the best file to use is the `~/.bashrc` file.

- Run `cd` to move to your `/home/instructor/` directory.
- Run `ls -la` to show all your files.

Notice the `.bashrc` file.

Unset

```
drwx----- 26 user user 4096 Sep  4 20:57 .
drwxr-xr-x  3 root root 4096 Aug 27 14:03 ..
-rw-----  1 user user 6779 Sep  4 21:48 .Bash_history
-rw-r--r--  1 user user  220 May 15  2017 .Bash_logout
-rw-r--r--  1 user user 3690 Aug 28 18:44 .bashrc
-rw-r--r--  1 user user  675 May 15  2017 .profile
```

If we want the alias to remain across logins, all we need to do is open the `~/ .bashrc` file and add them there.

Important: Before we edit this file, we should make a copy of it in case we make a mistake.

- Run `cp .bashrc .bashrc.bak`.
 - The `.bashrc` file will already have many configurations inside it, the scope of which lies outside this course.
 - All of the existing configurations can be ignored and you can add your aliases to the bottom of the file or the section commented for aliases.
- Run `nano .bashrc`.
 - Scroll down and look at the section that already has some aliases defined. These are some of the aliases we saw earlier:

Unset

```
# some more ls aliases
alias ll='ls -a1F'
alias la='ls -A'
alias l='ls -CF'
```

We can add aliases here, or modify the ones that already exist. Alternatively, you can create your own alias section at the bottom of the file.

Move to the bottom of the page and enter your alias along with a `# Custom Alias Section` comment:

Unset

```
# Custom Alias Section
alias la='ls -lah'
```

Save and close the file.

In order for the new setting to be loaded, we either have to reload the `~/ .bashrc` file, or we need to open a new terminal.

If we want to simply reload the file, we can use the `source` command.

- Run `source .bashrc` to demonstrate reloading `.bashrc`..
- Run `la` to show that your alias is working.

Close and reopen the terminal.

- Run `la` to show that your alias is still working.

Adding an Alias to `.bashrc`

Finally, in keeping with today's theme of becoming more efficient in the command line, we will add an alias to their `.bashrc` with one command.

Reloading the `.bashrc` can have its own alias, so we will create one for it now.

```
echo "alias rr='source ~/.bashrc'" >> ~/.bashrc && source ~/.bashrc
```

- `echo` sends what comes next directly to output.
- `alias` is our alias declaration.
- `rr` is our custom command that we use to reload the `.bashrc` file quickly.
- `'source ~/.bashrc'` is our command that reloads `.bashrc` and will be tied to `rr`.
- `>>` appends this to a file that we specify.
- `~/.bashrc` is the file we want to add our alias to.
- `&&` means that if the first command is completed successfully, it will run the command that comes next.
- `source ~/.bashrc` reloads the `~/.bashrc` file to enable our new `rr` alias.

If we wanted to, we could just use `echo "alias rr='source ~/.bashrc'" >> ~/.bashrc` to add the alias, and then reload the `.bashrc` file using `source ~/.bashrc`, but here we are using `&&` to complete it with one command.

From now on, you can just type `rr` to reload the `~/.bashrc`.

One more example of adding an alias this way:

- You want the `rm` command to always give you a warning before removing a file. Reading the man pages will reveal that the `-i` flag does this. So, we want the `rm` command to always default to `rm -i`.

```
echo "alias rm='rm -i'" >> ~/.bashrc && rr
```

- `echo` sends what comes next directly to output.
- `alias` is our alias declaration.

- `rm` is the alias we are using for our new, modified `rm` command.
- `'rm -i'` is our modified `rm` command that we want to use every time we type `rm`.
- `>>` appends this to a file that we specify.
- `~/.bashrc` is the file we want to add our alias to.
- `&&` means that if the first command is completed successfully, it will run the command that comes next.
- `rr` is the alias we created a moment ago for `source ~/.bashrc`.
- Run `tail -4 ~/.bashrc` to see the bottom of the file:

```
Unset
$ tail -4 ~/.bashrc
fi
fi
alias rr='source ~/.bashrc'
alias rm='rm -i'
```

05. Activity: Creating Aliases

- [Activity File: Creating Aliases](#)

06. Activity Review: Creating Aliases

07. Break

08. Creating Bash Scripts

We will now create short Bash scripts that use variables and command expansion.

A **Bash script** is a file containing a sequence of commands that is executed when the script is run.

- Bash scripting is very common among system administrators to automate common tasks.
- Creating a Bash script and then scheduling it to run at a regular time using `cron` is considered to be a basic ability of any system administrator.

In the following demo, we will cover **variables** and **command expansion** before putting it all together into a script.

Introduction to Variables

In computer programming, a variable is a location that stores some kind of data.

- We can think of it as a box that holds something so that you can refer to it later.
- If you no longer need what is in the box, you can overwrite its contents with new contents.

Variables can be overwritten and reused for different purposes. In other words, the data inside them may vary, hence the name *variable*.

- For our purposes, we will use a variable to hold either a number or a string of characters.
- Another common use of variables in a Bash script might be to hold the value of a file path.

Variable Demo

Log into the lab environment with the username `sysadmin` and password `cybersecurity` and open a terminal.

Let's make a variable for the `/etc/passwd` file path:

- Run `my_variable='/etc/passwd'`.
- Run `echo $my_variable`.

Your output should look like:

```
Unset
$ my_variable='/etc/passwd'
$ echo $my_variable
/etc/passwd
```

- `my_variable` is the name of the variable you want to create.
- `=` assigns your variable a value.
- `'/etc/passwd'` is the value that your variable holds.

A few more syntax-related notes:

- There must not be any spaces on either side of the `=` or you will get an error.

- Quotations must be used for any strings that are stored in a variable, particularly if there are spaces between characters.

When calling on a variable, it must be preceded with a `$`.

Run `num=5`.

Run `echo $num`.

- Your output should be:

Unset

```
$ num=5
$ echo $num
5
```

Built-In Variables

Bash has a number of built-in variables called **environment variables**. They are also known as **shell variables**.

They are always defined with all uppercase letters. For example, `$PWD` is an environment variable that returns the `pwd` command.

- Run `echo $PWD`.
Your output should be:

Unset

```
$ echo $PWD
/home/sysadmin
```

These can be creatively used to generate useful output:

- Run `echo "My present working directory is $PWD."`.
Your output should be:

Unset

```
$ echo "My present working directory is $PWD."
My present working directory is /home/sysadmin.
```

Run the following commands to see some built-in variables:

- `echo "My name is $USER"`: Provides the user name of the current user.
- `echo "My home directory is $HOME"`: Provides the home folder of the current user.
- `echo "The name of my computer is $HOSTNAME"`: Provides the name of the computer.
- `echo "My type of computer is a $MACHINE"`: Provides the type of computer.
- `echo "My user ID is $UID"`: Provides the UID of the current user.

Common expansion

Now we will move on to command expansion. Remember this command?

- ```
file $(find / -iname '*.txt' 2>/dev/null) > ~/Desktop/text\ files ;
tail ~/Desktop/text\ files
```

**Expansion** in Bash refers to any time something on the command line expands or morphs into something else.

- In the above command, the `find` command between the `$()` runs before any other part of the command.
- This `find` command *expands* into a list of items that it found. The rest of the commands are acting on that list, not acting on the `find` command itself.

Bash syntax uses the `$()` for command expansion.

- You can put any amount of commands chained together inside these brackets.
- Bash reads that chunk as whatever is returned from running the commands inside it.
- Then, the rest of the commands on the line run.

This is quite helpful when writing a script if we want one command to run before another command. To do this, we just surround that command with `$()`.

The `$` is similar to using it with a variable, but in this case we are receiving the output of the command.

For example, type `echo "The files in this directory are: $(ls)"`.

- `echo` sends what comes next to output.
- `"The files in this directory are: "` is sent directly to output and creates a headline.
- `$()`: Run *this command before* any other command on this line. In this case, it says "Run the command inside these brackets *before* running the `echo` command."
- `ls` is the command that runs first.

Run `echo "The files in this directory are: $(ls)"`.

Your output should be similar to:

Unset

```
echo "The files in this directory are: $(ls)"
The files in this directory are:file1
file2
file3
file4
...
```

Notice lack of line break before `file1`. We can fix this if we use a line break with `echo`.

- To use a line break, we need to use the `-e` flag with `echo` and then place the line break `\n` where we want it.
- Type `echo -e "The files in this directory are: \n$(ls)"`.

Your output should be similar to:

Unset

```
$ echo -e "The files in this directory are: \n$(ls)"
The files in this directory are:
file1
file2
file3
file4
...
```

## Variables in Scripts Demonstration

Now we will demonstrate how to use these concepts in a script.

Bash script files often end in `.sh` to indicate they are a `shell script`. However, a script file will still run with any extension.

- As an aside, Linux generally ignores file extensions. Instead, it looks at the contents of the file to determine how to use it. Therefore, you can create text files without any extension at all, but it is best practice to use the `.sh` file if you think other users may interact with your script.

To create a Bash script, it is important to use a text editor that does not add any extra formatting to the file when you save it. Some common options that text editors use in the command line are `nano`, `vim`, and `emacs`.

- In this class, we will stick with `nano` but you are encouraged to explore the other text editors if you are interested.

Begin by creating an empty file:

- Type `nano my_script`.
- Type `echo "Hello World."`.

Save and close the script.

If we tell Bash what shell to use to execute this file, it can be interpreted as a script.

- Run `Bash my_script`.

Your output should look like:

```
Unset
$ Bash my_script
Hello World.
```

While this format works, it is customary to use the `.sh` file extension to easily identify a script.

- Run `mv my_script my_script.sh`.

In the interest of efficiency, we can create a script file that will always run with `Bash`, so we don't have to type `Bash my_script.sh` every time we want to use it.

- To do this, at the top of the file, we add a line that starts with `#!` followed by the path of the shell we want the system to use.
- This line tells the system what shell we want to interpret this file.
- `#!` is often referred to as '**Hash Bang**' or '**Shebang**'.

Before we use the hash bang, we need to know the path of the Bash.

- Run `which Bash` to get the path to Bash.  
Your output should be:

```
Unset

$ which Bash
/bin/Bash
```

- 

**Note:** Running `Bash my_script.sh` is the same as `/bin/Bash my_script.sh`. `Bash` automatically knows what you mean by `Bash`. We will cover it more in the next part of the lesson (the `$PATH` variable).

Now we can add the line at the top of the file.

- Run `nano my_script.sh`.
- Above the `echo` line, add `#!/bin/Bash`:

Unset

```
#!/bin/Bash
echo "Hello World."
```

- - `#!` to indicate that what comes next is the shell we want to use to interpret this file.
  - `/bin/Bash` is the shell we want to use.
  - `echo "Hello World."` is the first line in our script.

Save and close the file.

Before we can run the script, we have to change its permissions to be an executable file.

- Run `ls -l my_script.sh`.  
Output should look like:

Unset

```
-rw-r--r-- 1 user user 20 Sep 5 16:20 my_script.sh
```

The file is not executable, so it can't be run on its own.

- Run `chmod +x my_script.sh`.
- Run `ls -l my_script.sh`.  
Your output should look similar to:

Unset

```
-rwxr-xr-x 1 user user 20 Sep 5 16:20 my_script.sh
```

Now, we can run the file on its own. The system will know to look inside the file for the `#!` line and interpret the file using the `/bin/Bash` program.

In order to run the file at this time, we only need to tell the system that the file is located in our current directory.

- Run `./my_script.sh`.
  - `./` is used to tell the system, "Execute the file that follows from *this* directory."

- Your output should be:

Unset

```
$./my_script.sh
Hello World.
```

In the event that a machine doesn't have the `Bash` program located at `/bin/Bash` or if it is using a different version of `Bash` in another location, this script may fail.

If we want our script to move around to different machines, we can use the line: `#!/usr/bin/env Bash`.

- `/usr/bin/env` will find the version of a program that the system is configured to use. When we use it with `Bash` we are saying to use the `Bash` configured on this system to interpret this file.
- `/usr/bin/env Bash` is important for you to understand, but for our purposes, using `/bin/Bash` is just fine.

## Quick Script Demonstration

Now we will create a short script to demonstrate how scripting works.

- We will intentionally keep this script to a series of commands in a list.

Open the script:

- Run `nano my_script.sh`.
- Enter this script:

Unset

```
#!/bin/Bash
name='Jake'
echo "Hello $name."
echo -e "\nThis is my script.\n"
echo -e "The files in $PWD are: \n$(ls)"
echo -e "\nCopying the passwd file to your current directory.\n"
cp /etc/passwd $PWD
echo -e "The files in $PWD are now: \n$(ls)"
echo " "
echo "Have a great day!"
```

Add or remove commands here as you see fit and then save and exit nano.

Next, run `./my_script.sh`.

- Your output should be similar to:

```
Unset
```

```
Hello Jake.
```

```
This is my script.
```

```
The files in /home/sysadmin are:
```

```
file1
```

```
file2
```

```
file3
```

```
...
```

```
Copying the passwd file to your current directory.
```

```
The files in /home/sysadmin are now:
```

```
file1
```

```
file2
```

```
file3
```

```
passwd
```

```
...
```

```
Have a great day!
```

---

## 09. Activity: Sysadmin Bash Script

- [Activity file: Sysadmin Bash Script](#)

---

## 10. Activity Review: First Bash Script

---

## 11. Custom Commands

Next, we will create a custom command that runs our script.

- This requires a bit of knowledge of what happens behind the scenes when you run a command and a built-in variable called the `PATH` variable.

In this demonstration, we will learn what the `PATH` variable is and how to customize it to create scripts that become your own custom commands.

### PATH Demonstration

Log into the lab environment with the username `sysadmin` and password `cybersecurity`.

- Open up a terminal and return to your home folder.

We know that every command we type is actually a program that runs. Those programs are stored in various directories like `/bin` and `/usr/bin`.

But, when you type a command, how does Bash know where that program is located?

If you were to make a copy of one of those programs and modify it, how would Bash know whether to use your new copy, or the old one?

Bash makes this decision by looking at the `$PATH` variable.

- `echo $PATH`

Unset

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/snap/bin:/home/user/.local/bin
```

The sole purpose of this environment variable is to hold a list of directories.

- When you type a command, Bash searches through this list for the program in order from left to right.
- When Bash finds the program, it uses it and stops searching.

To find our current path, we type `ls`. First, Bash searches for the `ls` program in the `/usr/local/bin` directory. If it isn't there, it searches the `/usr/bin` directory and so on down the list.

- If the program is not found in any of the directories in the `$PATH` variable, Bash will return 'Command Not Found.'
- Because Bash searches these directories in order, if we have 2 versions of a program, Bash will run the first one it finds.
- Since `$PATH` is just a variable, we can easily change it. We can add new directories for Bash to search, or even remove directories we don't want Bash to search.
- If we had a `scripts` directory full of custom scripts we wanted to use as commands, we only need to add that `scripts` directory to our `$PATH` and those scripts can then be run directly.

Create a `scripts` directory and add it to your `$PATH`:

- Run `mkdir my_scripts`.
- Run `mv sys_info.sh my_scripts/`.
- Run `ls my_scripts`.

Your terminal should resemble:

Unset

```
$ mkdir my_scripts
$ mv sys_info.sh my_scripts/
$ ls my_scripts
sys_info.sh
```

Now all we need to do is add our `my_scripts` directory to our `$PATH` so that Bash will find it when it searches for commands.

If we want to assign a value to a variable in Bash, we use the `VAR=VALUE` syntax.

In this case, to add a directory to our `$PATH`, we want to assign `PATH` to all of the directories it already has, *plus* our new directory.

- Run `echo $PATH` again.

Unset

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/snap/bin:/home/user/.local/bin
```

Copy the output with the right click.

- Type  
`PATH=/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/snap/bin:/home/user/.local/bin.`

Add your `my_scripts` directory.

- Type  
`PATH=/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/snap/bin:/home/user/.local/bin:/home/sysadmin/my_scripts.`

This will overwrite our `$PATH` variable with all of the directories it currently has, plus our new directory.

To make this easier, we can use the `$PATH` variable instead of copying its contents.

- Run `PATH=$PATH:/home/sysadmin/my_scripts.`
- Run `echo $PATH.`

You should see your appended path to the output:

```
Unset
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/snap/bin:/home/user/.local/bin:/home/sysadmin/my_scripts
```

Now run `sys_info.sh` directly:

```
Unset
$ sys_info.sh
```

If we wanted to make this a shorter command, we can shorten the length of the script name.

- Run `cd my_scripts.`
- Run `mv sys_info.sh sys.`
- Run `sys.`

### Saving `PATH` to our `.bashrc`.

Just like creating aliases, variables are only good like this for the duration of our session. Once your window is closed, your `$PATH` will return to its default.

The good news is that we can save the path `PATH=$PATH:/home/sysadmin/my_scripts` to our `.bashrc!`.

- Type `echo "PATH=$PATH:/home/sysadmin/my_scripts" >> ~/.bashrc.`

This is exactly how we added aliases to our `.bashrc` previously.

- Run `nano .bashrc`.
- Move to the bottom of the file and enter the new `PATH` variable.

Unset

```
export PATH=$PATH:/home/sysadmin/my_scripts
```

Here we want to use `export` to make this variable to *all* processes across the system. If you don't use `export`, your `$PATH` variable may not always work.

Save and quit `nano`.

- Run your alias for reloading the `.bashrc` file.
- Run `rr`.
- Run `echo $PATH` to show your updated `PATH`.

Creating custom scripts that you can use as custom commands like this is a valuable and useful skill to have at the command line.

---

## 12. Activity: Custom Commands

- [Activity File: Custom Commands](#)

---

## 13. Activity Review: Custom Commands

---